

Brac University
Department of Electrical & Electronic Engineering
Fall2025



Course Number : EEE282

Section : 01

Group No : 04

Lab Report

Experiment no: 03

Name of the experiment:

Numerical Differentiation Techniques

Prepared by:

Name: Alif Tamjid ID: 24121308

All Group members:

<i>Sl.</i>	<i>ID</i>	<i>Name</i>
1.	24121205	Souvik Barman Ratul
2.	24121308	Alif Tamjid
3.	23221082	Sabrina Maliha
4.	24121019	Monazzi Siddique

Objective:

To learn and use numerical differentiation methods in MATLAB to estimate derivatives of functions and compare the accuracy of different finite difference methods.

Theoretical Background:

Numerical differentiation uses the Taylor Series to estimate derivatives by using function values at nearby points and ignoring higher-order terms.

The Forward Difference Method estimates the slope using the current point and the next point. It is simple but not very accurate, and its error depends on the step size h .

$$f'(x) = \{f(x + h) - f(x)\}/h$$

The Backward Difference Method uses the previous point instead and gives similar accuracy, mainly useful near the end of data.

$$f'(x) = \{f(x) - f(x - h)\}/h$$

The Central Difference Method uses points on both sides of the point, making it more accurate because errors cancel out. A smaller h gives better results.

$$f'(x) = \{f(x + h) - f(x - h)\}/2h$$

A more accurate approach is the Fourth-Order Central Difference Method, which uses more surrounding points and significantly reduces error.

In this experiment, the higher-order central difference method is used for better accuracy. The numerical result is compared with the exact derivative obtained using MATLAB's symbolic differentiation, and the small difference confirms the reliability of the numerical method.

Problem Statement:

Problem-1: Derive the expressions of Forward Divided Difference (FDD), Backward Divided Difference (BDD), Central Divided Difference (CDD) of 1st order derivatives.

Answer:

▣ Forward Divided Difference (FDD).

Taylor Series:

$$f(x+h) = f(x) + f'(x) \frac{h}{1!} + f''(x) \frac{h^2}{2!} + f'''(x) \frac{h^3}{3!} + \dots$$

$$\Rightarrow f'(x) \frac{h}{1!} = f(x+h) - f(x) - f''(x) \frac{h^2}{2!} - f'''(x) \frac{h^3}{3!} + \dots$$

$$\Rightarrow f'(x) = \frac{f(x+h) - f(x)}{h} + 0 \quad \left[\begin{array}{l} \text{Here, } h \text{ is very small.} \\ \text{For that reason it gives} \\ \text{a small value, close to zero} \end{array} \right]$$

$$\therefore f'(x) = \frac{f(x+h) - f(x)}{h}$$

▣ Backward Divided Difference (BDD)

Taylor Series

$$f(x-h) = f(x) - f'(x) \frac{h}{1!} + f''(x) \frac{h^2}{2!} - f'''(x) \frac{h^3}{3!} + \dots$$

$$\Rightarrow f'(x) \frac{h}{1!} = f(x) - f(x-h) + f''(x) \frac{h^2}{2!} - f'''(x) \frac{h^3}{3!} + \dots$$

$$\therefore f'(x) = \frac{f(x) - f(x-h)}{h} + 0$$

$$\therefore f'(x) = \frac{f(x) - f(x-h)}{h}$$

▣. central divided difference (CDD)

$$f(x+h) + f(x-h) = 2f'(x)\frac{h}{1!} + 2f'''(x)\frac{h^3}{3!} + 2f^{(5)}(x)\frac{h^5}{5!} + \dots$$

$$\Rightarrow \frac{f(x+h) - f(x-h)}{2} = f'(x)h + f'''(x)\frac{h^3}{3!} + \dots$$

$$\therefore f'(x) = \frac{f(x+h) - f(x-h)}{2h} + O(h^2)$$

$$\therefore f'(x) = \frac{f(x+h) - f(x-h)}{2h}$$

Problem-2: Write the MATLAB functions for FDD, BDD & CDD of 1st order derivatives using the expressions you derived in the previous problem.

Answer:

Function being Code:

(FDD Part)

```
function d = Problem2_Code1func(f, x, h)

    d = (f(x+h) - f(x)) / h; %%Forward Divided Difference

end
```

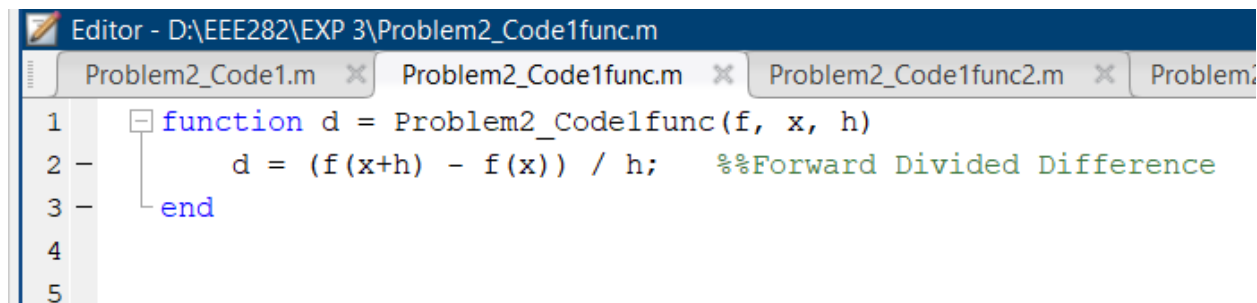


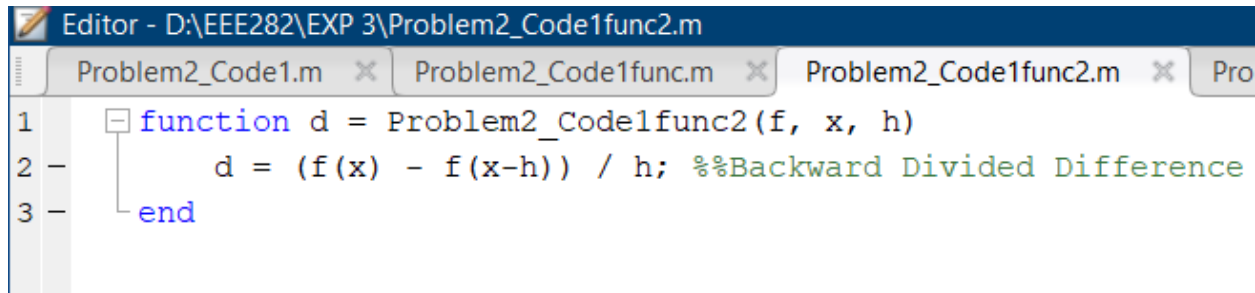
Figure 1: Code of the function that is being called for FDD

(BDD Part)

```
function d = Problem2_Code1func2(f, x, h)

d = (f(x) - f(x-h)) / h; %%Backward Divided Difference

end
```



```
Editor - D:\EEE282\EXP 3\Problem2_Code1func2.m
Problem2_Code1.m x Problem2_Code1func.m x Problem2_Code1func2.m x Pro
1 function d = Problem2_Code1func2(f, x, h)
2     d = (f(x) - f(x-h)) / h; %%Backward Divided Difference
3 end
```

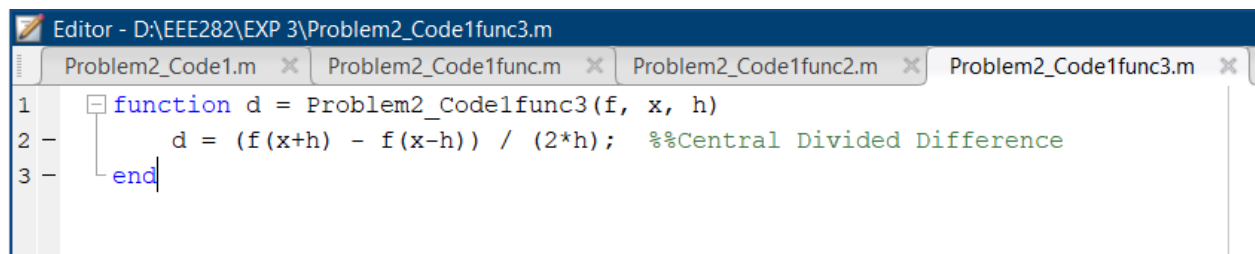
Figure 2: Code of the function that is being called For BDD

(CDD Part)

```
function d = Problem2_Code1func3(f, x, h)
```

```
    d = (f(x+h) - f(x-h)) / (2*h); %%Central Divided Difference
```

```
end
```



```
Editor - D:\EEE282\EXP 3\Problem2_Code1func3.m
Problem2_Code1.m x Problem2_Code1func.m x Problem2_Code1func2.m x Problem2_Code1func3.m x
1 function d = Problem2_Code1func3(f, x, h)
2     d = (f(x+h) - f(x-h)) / (2*h); %%Central Divided Difference
3 end
```

Figure 3: Code of the function that is being called For CDD

Main Script:

```
clear all;
```

```
close all;
```

```
clc;
```

```
f = @(x)(x.^2 - 3*x);
```

```
x = 5;
```

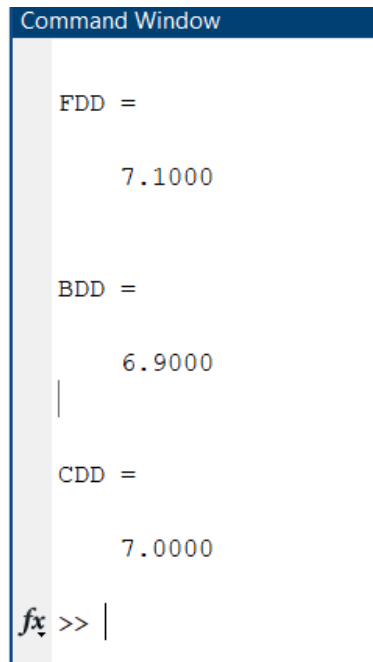
```
h = 0.1;
```

```
FDD = Problem2_Code1func(f, x, h)
```

```
BDD = Problem2_Code1func2(f, x, h)
```

```
CDD = Problem2_Code1func3(f, x, h)
```

Output:



```
Command Window

FDD =

    7.1000

BDD =

    6.9000

CDD =

    7.0000

fx >> |
```

The image shows a MATLAB Command Window with a dark blue title bar. The window contains the following text: 'FDD =' followed by '7.1000' on the next line, 'BDD =' followed by '6.9000' on the next line, and 'CDD =' followed by '7.0000' on the next line. At the bottom, there is a prompt 'fx >> |' with a vertical cursor line.

Figure 4: Result of Forward Difference method, Backward difference method and Central Difference method.

Explanation:

The function is simple because we are only finding first-order derivatives. Finite difference methods estimate derivatives using function values at nearby points separated by a step size h . The Forward Difference method uses the current point and the next point, while the Backward Difference method uses the current point and the previous point. The Central Difference method is more accurate because it considers points on both sides of the current point. We validate our functions by comparing them with a separate main script.

Objective :

The aim of this problem is to determine the first derivative of a given function at a specific point ($x=3.5$) by using numerical differentiation methods and to compare the result with the analytical solution.

Theoretical Background :

Problem Statement :

Problem-3: For the given function

$$f(x) = \frac{2x \sin(3x) + e^{-2x}}{x^{0.5}}$$

Determine the value of the 1st derivative of the function numerically for $x = 3.5$ with best possible accuracy.

Also, compare the numerical results of the derivatives with the analytical (by using the formulae of differentiation) results.

Answer:

Code:

```
clear all;
```

```
close all;
```

```

clc;

x=3.5;

f = @(x) (2*x.*sin(3*x) + exp(-2*x)) ./ (x.^0.5);

h=0.001;

FDD = (f(x+h)-f(x))/h; %%Forward Divided Difference

BDD = (f(x)-f(x-h))/h; %%Backward Divided Difference

CDD = (f(x+h)-f(x-h))/(2*h); %%Central Divided Difference

fprintf("By using numerical method,Forward Difference method value is
%.5f.\n",FDD);

fprintf("By using numerical method,Backward Difference method value is
%.5f.\n",BDD);

fprintf("By using numerical method,Central Difference method value is
%.5f.\n",CDD);

f_a=@(x) (((2*sin(3*x))+(6*x*cos(3*x))-(2*exp(-2*x)))*(sqrt(x)))-(0.5*((2*x*sin
(3*x))+exp(-2*x))*(1/sqrt(x))))/x;% Formula for analytical method.

Y = f_a(x);

fprintf("By using analytical method, the value is %f.\n",Y);

Error_FDD = abs(FDD-Y);%Error in Forward Difference method with respect to
analytical result.

Error_BDD = abs(BDD-Y);%Error in Backward Difference method with respect to
analytical result.

```

```
Error_CDD = abs(CDD-Y);%Error in Central Difference method with respect to analytical result.
```

```
fprintf("Error in Forward difference method %f.\n",Error_FDD);%Showing the value.
```

```
fprintf("Error in Backward difference method %f.\n",Error_BDD);%Showing the value.
```

```
fprintf("Error in Central difference method %f.\n",Error_CDD);%Showing the value.
```

```
%Comparing which process gives the least error and showing that the least error giving process give best possible accuracy.
```

```
if Error_FDD<Error_BDD && Error_FDD<Error_CDD
```

```
fprintf(" The Forward Difference method shows the best possible accuracy.\n");
```

```
elseif Error_BDD < Error_FDD && Error_BDD < Error_CDD
```

```
fprintf(" The Backward Difference method shows the best possible accuracy.\n");
```

```
elseif Error_CDD < Error_FDD && Error_CDD < Error_BDD
```

```
fprintf("The Central Difference method shows the best possible accuracy.\n");
```

```
end
```

Output:

```
Command Window
New to MATLAB? See resources for Getting Started.
By using numerical method,Forward Difference method value is -5.79506.
By using numerical method,Backward Difference method value is -5.82322.
By using numerical method,Central Difference method value is -5.80914.
By using analytical method, the value is -5.809150.
Error in Forward difference method 0.014094.
Error in Backward difference method 0.014074.
Error in Central difference method 0.000010.
The Central Difference method shows the best possible accuracy.
>>
```

Figure 5: Output showing why Central Difference Method shows the best accuracy.

Explanation:

At first clear all, close all, clc commands are used to remove any previous data execution command affecting the current script. Then the given function is written.

A small step size $h=0.001$ is taken to reduce error. The value of x at 3.5 is calculated by using the three formulas for three methods- Forward difference method, Backward Difference method and Central Difference method and shown using the “fprintf” function. By using the quotient rule of the two function of first

derivative, the formula of $\frac{d(u/v)}{dx} = \frac{v \cdot \frac{du}{dx} - u \cdot \frac{dv}{dx}}{v^2}$ is applied. Using the formula,

function value (Y) when $x=3.5$ is found out analytically. Then error is found by comparing every method with respect to the analytical method. Finally, using the “ if ” condition, the method which gives the least error with respect to the analytical method is the one that gives the best possible accuracy.

Discussion:

The given function is written and then its value at $x=3.5$ is calculated using the forward difference method, central difference method and backward difference method having a small step size. Then a function is written to calculate the value of the given function by using an analytical method. For analytical methods, the quotient rule of differentiation is used. Error is calculated by comparing each of the values found using numerical methods with analytical methods. Using the “if” condition to compare which method gives least error from analytical value, the result is presented saying “The central difference method shows the best possible accuracy.”

Objective:

The aim of this experiment is to determine the velocity of the object at a time by differentiating displacement-time data numerically.

Theoretical Background:**Problem Statement :**

Problem-4: The following data are provided for the displacement, s of a moving object as a function of time, t :

s (meter)	0	4	8	12	16	20
t (sec.)	0	34.7	61.8	82.8	99.2	112.0

Determine the velocity, v of the object for each of the time instants.

[Just to recall: $v = \frac{ds}{dt}$.]

Answer:

Code:

```
clear all;

close all;

clc;

S = [ 0 4 8 12 16 20];%Forming array with the values of S.

t = [0 34.7 61.8 82.8 99.2 112];%Forming array with the values of t.

leng_s = length(S);%Measuring number of elements in the array.

v = zeros (1,leng_s);%allocating array to store the values of velocity

%FDD = (f(x+h)-f(x))/h    Forward Divided Difference

%BDD = (f(x)-f(x-h))/h    Backward Divided Difference

%CDD = (f(x+h)-f(x-h))/(2*h)    Central Divided Difference

for i=1:leng_s

if i ==1 % Forward difference method used because no point exist before this
first number.

v(i) = (S(i+1)-S(i))./(t(i+1)-t(i));
```

```

elseif i>1 && i<=(leng_s - 1)% Central difference method used because points
exist on both sides.

v(i) = (S(i+1)- S(i-1))./(t(i+1)-t(i-1));

elseif i==leng_s % Backward difference method used because no point exists
after this last number.

v(i) = (S(i) - S(i-1))./(t(i) - t(i-1));

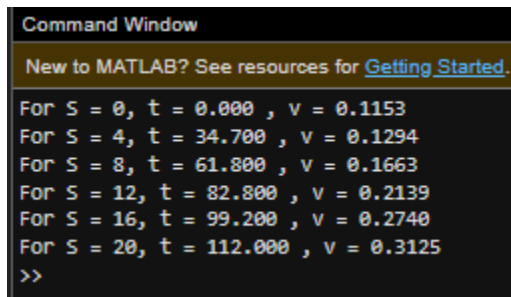
end

fprintf("For S = %d, t = %.3f , v = %.4f\n",S(i),t(i),v(i)); %showing every
value of velocity for every value of displacement and time.

end

```

Output:



Command Window

New to MATLAB? See resources for [Getting Started](#).

```

For S = 0, t = 0.000 , v = 0.1153
For S = 4, t = 34.700 , v = 0.1294
For S = 8, t = 61.800 , v = 0.1663
For S = 12, t = 82.800 , v = 0.2139
For S = 16, t = 99.200 , v = 0.2740
For S = 20, t = 112.000 , v = 0.3125
>>

```

Figure 6: Showing the value of Velocity v for every value of displacement and time.

Explanation:

At first, clear all, close all, clc commands are used to remove any previous data execution command affecting the current script. The given values are stored in arrays. Length of the array of displacement is measured. An array (v) containing zero is allocated to contain the values of velocity for every value of displacement and time. A “for” loop is run for every value of displacement and time. For the first value of S and t, the forward difference method formula is used since we don't have any previous point. For the last value of S and t, the backward difference method formula is used since we don't have the seventh number point. For the point in array number 2 to 5 , the central difference method formula is used. After every loop, the value of v for the corresponding values of displacement and time is stored in the array ‘ v ’. Finally the result is shown using the “fprintf” function.

Discussion:

Velocity is the rate of change of displacement, that is, the first derivative of position (or displacement) with respect to time. A loop is run for every given value of displacement and time. The forward difference method and the backward difference method is used for the first and the last value of displacement and time

respectively. For values of displacement and time for the 2nd to the 5th, the Central difference method is used. Finally, all the values are shown as output.

Lab Task 01:

Lab Task 1:

Write the MATLAB code for numerical differentiation using Forward Difference Method.

Given $f(x) = e^x$, find $f'(1)$ using $h=10^{-1}, 10^{-2}, \dots$, upto 10^{-6} . Find out the error in each case by comparing the calculated value with exact value.

Answer:

Code:

```
clear all;
```

```
close all;
```

```
clc;
```

```
% Given function
```

```
f = @(x) exp(x);
```

```
x = 1;           % Point of differentiation
```

```
exact = exp(x);    % Exact value of derivative
```

```
h_values = 10.^(-(1:6)); % h = 10^-1 to 10^-6
```

```
for i = 1:length(h_values)
```

```
    h = h_values(i);
```

```
    % Forward difference formula
```

```
    numerical = (f(x + h) - f(x)) / h;
```

```
    % Error calculation
```

```
    error = abs(exact - numerical);
```

```
    % Display results
```

```
    fprintf('the value for h = %.7f is %.3f and the error is %.3f\n', h, numerical,  
error);
```

```
end
```

Output:

Command Window

```
the value for h = 0.1000000 is 2.859 and the error is 0.141
the value for h = 0.0100000 is 2.732 and the error is 0.014
the value for h = 0.0010000 is 2.720 and the error is 0.001
the value for h = 0.0001000 is 2.718 and the error is 0.000
the value for h = 0.0000100 is 2.718 and the error is 0.000
the value for h = 0.0000010 is 2.718 and the error is 0.000
fx >> |
```

Explanation:

This MATLAB code demonstrates how to approximate the derivative of a function using the Forward Difference Method and compare it with the exact derivative. The function chosen is $f(x) = \exp(x)$, defined as an anonymous function, and the point of differentiation is set at $x=1$. The exact derivative at this point is also calculated as $\exp(1)$. A vector of h_values is created ranging from 10^{-1} to 10^{-6} to observe how the approximation changes with different sizes. The code then loops through each step size and applies the Forward Difference formula $f'(x) = \{f(x + h) - f(x)\}/h$, which estimates the slope using the current and next point. For each h , the error is calculated as the absolute difference between the exact derivative and the numerical result.

Lab Task 02:

Lab Task 2:

Write the MATLAB code for numerical differentiation using Central Difference Method.

Given $f(x) = \sin(\cos(1/x))$ find $f'(1/\sqrt{2})$ using $h=10^{-1}, 10^{-2}, \dots$ up to 10^{-6} . Find out the error in each case by comparing the calculated value with exact value.

Answer:

Code:

```
clear all;  
close all;  
clc;
```

```
% Given function
```

```
f = @(x) sin(cos(1./x));
```

```
x = 1/sqrt(2);          % Point of differentiation
```

```
exact = (sin(1./x).*cos(cos(1./x))) ./ (x.^2);    % Exact value of derivative
```

```
h_values = 10.^(-(1:6)); % h = 10^-1 to 10^-6
```

```
for i = 1:length(h_values)
```

```
    h = h_values(i);
```

```
    % Forward difference formula
```

```
    numerical = (f(x + h) - f(x - h)) / (2*h);
```

```
    % Error calculation
```

```
    error = abs(exact - numerical);
```

```
    % Display results
```

```
    fprintf('the value for h = %.7f is %.3f and the error is %.3f\n', h, numerical,  
error);
```

```
end
```

Output:

```
Command Window

the value for h = 0.1000000 is 1.981 and the error is 0.030
the value for h = 0.0100000 is 1.952 and the error is 0.000
the value for h = 0.0010000 is 1.952 and the error is 0.000
the value for h = 0.0001000 is 1.952 and the error is 0.000
the value for h = 0.0000100 is 1.952 and the error is 0.000
the value for h = 0.0000010 is 1.952 and the error is 0.000
fx >>
```

Explanation:

This MATLAB code demonstrates how to approximate the derivative of a function using the Forward Difference Method and compare it with the exact derivative. The function chosen is $f(x)=\sin(\cos(1/x))$, defined as an anonymous function, and the point of differentiation is set at $x=1/\sqrt{2}$. The exact derivative at this point is also calculated as $(\sin(1/x)\cos(\cos(1/x))) / (x^2)$. A vector of h_values is created ranging from 10^{-1} to 10^{-6} to observe how the approximation changes with different sizes. The code then loops through each step size and applies the central difference formula $f'(x) = \{f(x + h) - f(x - h)\}/2h$, which uses points on both sides of the point, making it more accurate because errors cancel out. For each h , the error is calculated as the absolute difference between the exact derivative and the numerical result.

Conclusion:

In this experiment, we learned how to estimate derivatives using various numerical differentiation techniques. By using Forward, Backward, and Central Difference methods, we observed differences in accuracy depending on how nearby function values are utilized. The Central Difference approach showed higher accuracy because it considers points on both sides of the target value. We also implemented a higher-order central difference formula in MATLAB, which produced results nearly identical to the exact analytical derivative. The small calculated errors confirmed the reliability of this method. Additionally, we noticed the impact of the step size h on the results, with smaller values providing more precise approximations. Finally, these methods were applied to practical data, such as calculating velocity from displacement and time measurements. Overall, the

experiment provided hands-on understanding of numerical differentiation and demonstrated how MATLAB can be used to apply these techniques effectively to solve real engineering problems.